

Circles

A Music Statistics & Social Platform

Software Development Case Study

Requirements Specification Document

Assignment 1 — Requirements Gathering & Specification

Course:	Software Development Case Study
Assignment:	1 — Requirements Engineering
Week:	Week 2
Group:	CSE-2507M
Team:	Alexandr Tyulkov
Date:	March 19, 2026

Table of Contents

1	Part 1 — Project Planning	2
1.1	System Description & Purpose	2
1.2	Target Audience	2
1.3	Problem Statement	2
1.4	Feasibility Study (TELOS Model)	2
1.5	Team	3
2	Part 2 — Requirements Documentation	4
2.1	Project Overview	4
2.2	Functional Requirements	4
2.2.1	Use Case Diagrams	5
2.3	Non-Functional Requirements	6
2.3.1	Performance	6
2.3.2	Usability	6
2.3.3	Security	6
2.3.4	Reliability	7
2.3.5	Scalability	7
2.4	Constraints & Assumptions	7
2.4.1	Constraints	7
2.4.2	Assumptions	7
2.4.3	Technology Stack (Fixed)	8
2.5	AI Tools Usage Log	8
3	Part 3 — Initial System Scope Diagram	9
3.0.1	Component Descriptions	9

1 Part 1 — Project Planning

1.1 System Description & Purpose

Circles is a web-based music statistics and social platform built on the Spotify API. It stores every played track as a “scrobble” and exposes that data through analytics dashboards, a social graph, and AI-generated insights.

Users can browse top artists, albums, and tracks across configurable time windows; view audio-feature profiles (energy, valence, danceability); follow other users; compete on leaderboards; request AI personality reports; manage playlists; and import historical listening data.

Circles draws on Last.fm (scrobbling) and Spotify Wrapped (periodic summaries), but makes statistics and social features available continuously — not just at year’s end.

1.2 Target Audience

The primary users of Circles are:

- **Music enthusiasts and power listeners** (ages 16–35) who want persistent statistics beyond what Spotify’s native app exposes.
- **Socially driven listeners** who compare tastes with friends and discover music through others’ activity.
- **Data-curious individuals** interested in genre evolution, mood trends, and listening streaks.

A secondary audience is **casual Spotify users** who join through a friend’s invite.

1.3 Problem Statement

Spotify provides end-of-year “Wrapped” summaries and a “Recently Played” list capped at 50 tracks. There is no built-in way to:

- Maintain a permanent, searchable listening history beyond the 50-track API limit.
- Track statistics over arbitrary time windows (e.g., “my top artists in the last 90 days”).
- See what friends are listening to or discover music through social connections.
- Get narrative-style insights about one’s own listening identity.

Last.fm covers scrobbling but has a dated UI and no AI features. Circles combines persistent scrobbling, analytics, social features, and AI profiling in one product.

1.4 Feasibility Study (TELOS Model)

Dimension	Assessment
Technical	All required technologies are mature and in active use across the stack. The Spotify Web API provides OAuth 2.0 and recently-played endpoints. PostgreSQL handles time-series scrobble data. Hono and React with TanStack cover the API and frontend layers. The Claude API provides generative text. The team has hands-on experience with each component.
Economic	A student/portfolio project with no commercial requirement. The Spotify and Claude APIs both offer free tiers sufficient for development. PostgreSQL and S3 run on free or low-cost cloud plans (\$10–20/month for a full deployment).
Legal	Spotify data is accessed only after explicit OAuth consent and is not redistributed. Only metadata (track IDs, timestamps) is stored — no audio content. User data

Dimension	Assessment
	handling is GDPR-aligned: minimisation, consent, right to deletion. AI-generated content is per-user and not persisted beyond a short cache window.
Operational	Browser-based, no installation required. OAuth tokens are refreshed automatically on a scheduled job before expiry. Scrobble syncing runs on a cron schedule. Local development uses Docker Compose; production uses standard cloud deployment.
Schedule	14-week semester project. Auth, scrobble storage, and basic analytics are the critical path (first four weeks). Social, AI, and import features follow in subsequent sprints.

1.5 Team

Name	Responsibilities
Alexandr Tyulkov	Backend API design, database schema, background worker, Spotify integration, AI features, frontend architecture, UI/UX, React components, TanStack Router, data visualisations

2 Part 2 — Requirements Documentation

2.1 Project Overview

Goal: A web application that gives Spotify users a permanent record of their listening history, with analytics, social interactions, playlist management, and AI-generated insights.

Main functionalities:

- Spotify OAuth authentication and automatic scrobble syncing
- Statistics dashboards — top artists, albums, tracks, genres across time ranges
- Audio-feature analysis and visualisation (radar charts, timeline)
- Social graph — follow/unfollow users, view friends' recent activity
- Global and time-windowed leaderboards
- Music-match score between two users based on shared listening
- AI-generated personality reports: Taste DNA, personalised roasts, scene reports
- Playlist creation, management, and auto-generation
- Historical data import via ZIP file upload
- Public user profiles with customisable usernames

Target audience: Music enthusiasts, social listeners, and data-curious Spotify users aged 16–35 (see Section 1.2).

2.2 Functional Requirements

US-01 As a **registered user**, I want to **connect my Spotify account via OAuth** so that my listening history is automatically tracked without manual input..

US-02 As a **user**, I want to **view my top artists, albums, and tracks across selectable time ranges (7 days, 30 days, 90 days, 1 year, all-time)** so that I can understand how my music taste changes over time..

US-03 As a **user**, I want to **follow other users and see a feed of their recent listening activity** so that I can discover new music through people whose taste I trust..

US-04 As a **user**, I want to **request an AI-generated report about my music personality (Taste DNA, roast, or scene report)** so that I receive a fun and personalised narrative about my listening identity..

US-05 As a **user**, I want to **create playlists, add or remove tracks, and have the system auto-generate a weekly playlist from my recent listening** so that I can organise and share my music collections without leaving the platform..

US-06 As a **user**, I want to **see a global leaderboard ranked by total scrobbles for the current week and all-time** so that I can compare my listening volume with the broader community..

US-07 As a **user**, I want to **upload a ZIP archive of my historical listening data** so that I can retroactively import listening history from before I joined Circles and have a complete statistical picture..

US-08 As a **user**, I want to **view a public profile for any other registered user, including their stats and top tracks** so that I can learn about someone's music taste before deciding to follow them..

US-09 As a **user**, I want to **see a 'music match' score between myself and another user** so that I can quantify how similar our tastes are and find people with overlapping preferences..

US-10 As a **user**, I want to **browse a 'time machine' view that shows what I was listening to on any past date** so that I can relive specific moments through the music I had on at the time..

2.2.1 Use Case Diagrams

Diagram A — Statistics, Social & Discovery

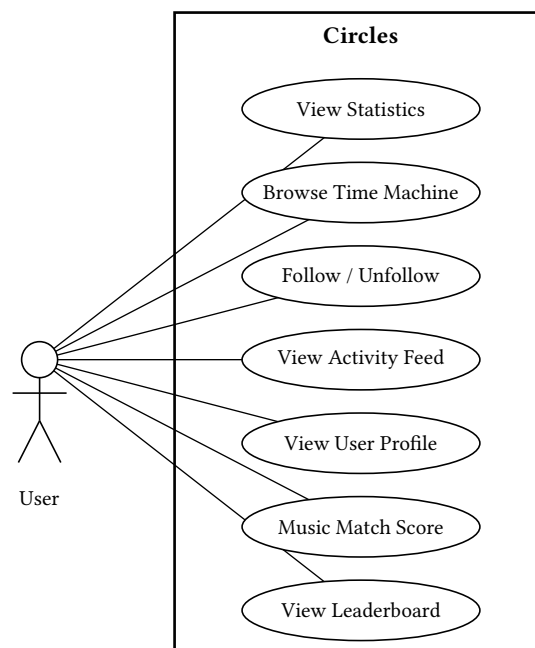
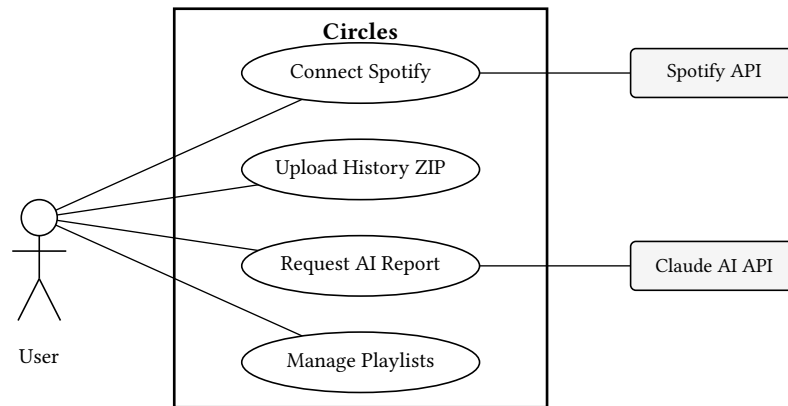


Diagram B — Auth, Import, AI & Playlists



2.3 Non-Functional Requirements

2.3.1 Performance

NFR-P1	The API must respond to 95% of requests within 200 ms under normal load (up to 100 concurrent users).
NFR-P2	The Spotify history sync workflow must complete for a single user within 30 seconds of being triggered.
NFR-P3	The leaderboard endpoint, which aggregates millions of rows, must respond within 500 ms by using server-side caching with a TTL of 5 minutes.
NFR-P4	Frontend initial page load (First Contentful Paint) must occur within 2 seconds on a standard broadband connection.

2.3.2 Usability

NFR-U1	The UI must be fully responsive and usable on screens with a minimum width of 375 px (iPhone SE equivalent).
NFR-U2	All interactive controls must meet WCAG 2.1 Level AA contrast ratios (4.5:1 minimum for normal text).
NFR-U3	A new user must be able to complete Spotify login and reach their first dashboard within under 60 seconds with no prior instruction.
NFR-U4	Error states (failed API calls, empty history) must surface a human-readable message — never a raw stack trace or HTTP status code.

2.3.3 Security

NFR-S1	All communication between the browser, API, and external services must use HTTPS/TLS 1.2+ . HTTP requests must be rejected or redirected.
NFR-S2	Spotify OAuth access tokens and refresh tokens must be stored encrypted at rest and never exposed to the client.
NFR-S3	All protected API routes must validate a session token on every request; unauthenticated requests must receive HTTP 401 .
NFR-S4	User data must not be accessible to other users unless the account is explicitly set to public .

2.3.4 Reliability

NFR-R1	The system must maintain $\geq 99.5\%$ monthly uptime for the web application (≤ 3.6 hours downtime/month).
NFR-R2	The token-refresh background job must run at least every 50 minutes to ensure Spotify tokens (valid for 60 minutes) never expire unexpectedly during a user session.
NFR-R3	Failed background jobs (sync, import) must be retried automatically up to 3 times with exponential back-off before being marked as failed.
NFR-R4	The system must handle Spotify API rate-limit responses (HTTP 429) gracefully by backing off and retrying without data loss.

2.3.5 Scalability

NFR-SC1	The database schema must support at least 10 million scrobble records across all users without query degradation (achieved via indexed <code>played_at</code> and <code>user_id</code> columns).
NFR-SC2	The background worker must be horizontally scalable — multiple worker instances must be deployable without duplicate job execution (guaranteed by the Hatchet job queue).
NFR-SC3	The API server must be stateless so that multiple instances can run behind a load balancer without session-affinity requirements.

2.4 Constraints & Assumptions

2.4.1 Constraints

Team	Two developers, each contributing approximately 10–15 hours per week for 14 weeks.
Spotify API	Access to recently-played tracks is limited to the last 50 plays per API call; full history reconstruction requires either iterative polling or a user-supplied data export.
Spotify API (rate limits)	The Spotify Web API imposes rate limits that prevent syncing more than a few hundred tracks per minute per user token.
AI cost	AI report generation is gated behind a per-user weekly cache bucket to limit Anthropic API spend to within the free-tier quota during development.
Time	The full feature set must be demonstrable by the end of the semester (approximately 14 weeks from project start).
Platform	Circles is a web-only application for the scope of this project. Native mobile apps are out of scope.

2.4.2 Assumptions

A1	All target users have an existing Spotify account; the system does not manage music playback or require a Spotify Premium subscription.
-----------	---

A2	Users grant the required Spotify OAuth scopes (user-read-recently-played, playlist-modify-public) during first login.
A3	The development environment uses Docker Compose to run PostgreSQL, the Hatchet job queue, and MinIO (S3-compatible storage) locally. A cloud equivalent is assumed for production.
A4	Internet connectivity is required; the application has no offline mode.
A5	Historical import files are in the Spotify Extended Streaming History JSON format (ZIP archive as provided by Spotify’s “Download your data” feature).

2.4.3 Technology Stack (Fixed)

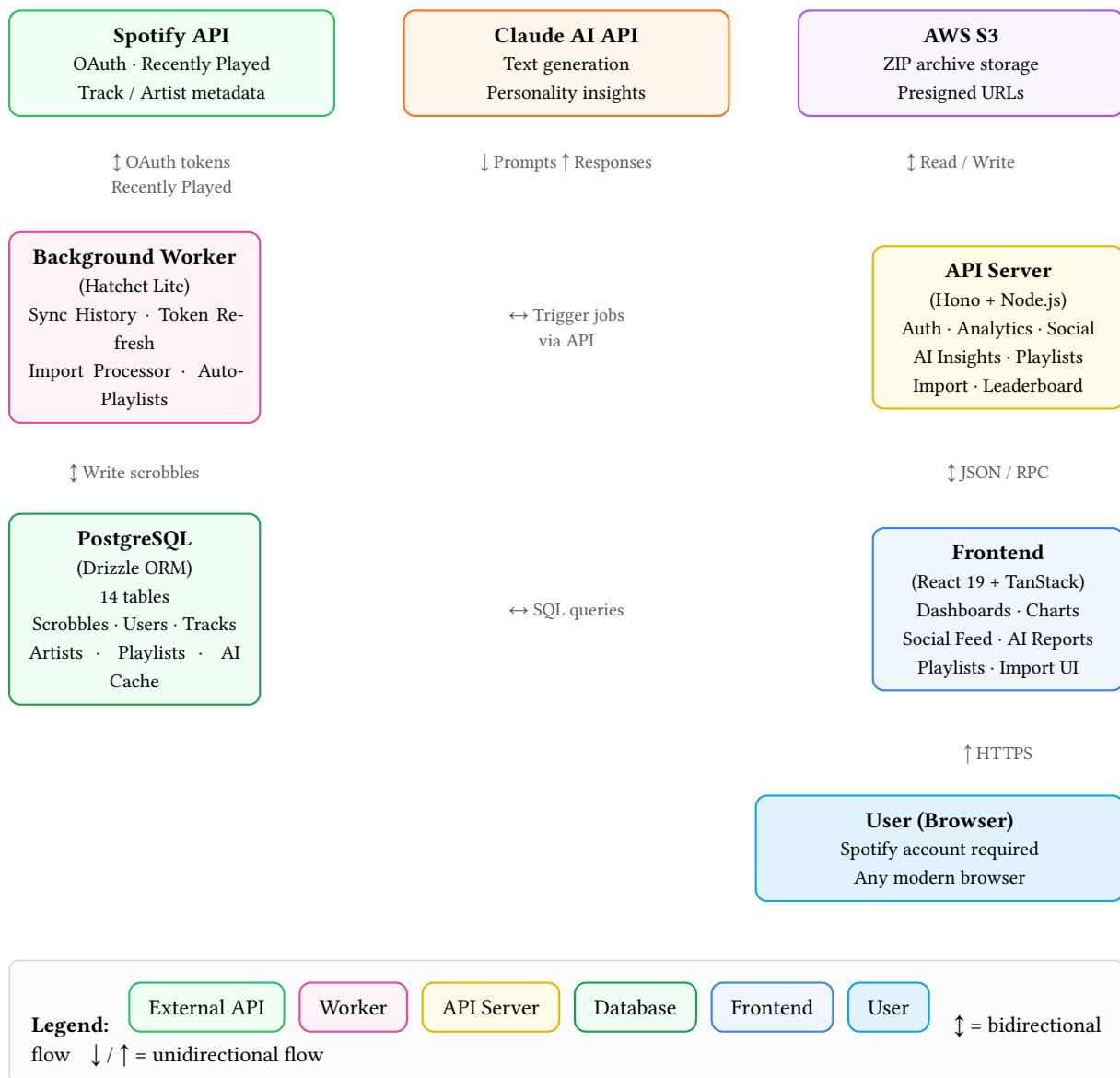
Frontend	React 19, TanStack Start (SSR), TanStack Router, TanStack Query, Tailwind CSS
Backend	Node.js ≥ 22, Hono (HTTP framework), better-auth (OAuth)
Database	PostgreSQL 16, Drizzle ORM
Background Jobs	Hatchet Lite (workflow orchestration)
AI	Anthropic Claude API
File Storage	AWS S3 (MinIO for local dev)
Build Tooling	Vite+ (vp CLI), pnpm workspaces

2.5 AI Tools Usage Log

Tool	Specific Purpose	What was kept / changed / rejected
Claude (claude-sonnet-4-6)	Brainstormed section structure, suggested candidate user stories and NFR wording, cross-checked requirements against the implemented codebase (schema, routes, worker workflows).	Kept structural suggestions and a few feature prompts that matched the project. All requirements were rewritten against the actual source code. NFRs were tightened with measurable thresholds. Suggestions that didn’t match scope (e.g. unimplemented features) were discarded or reworded.

3 Part 3 — Initial System Scope Diagram

Major subsystems, actors, and data/control flow between components.



3.0.1 Component Descriptions

Component	Responsibility
User (Browser)	Authenticates via Spotify OAuth and uses all platform features through a web browser.
Frontend (React + TanStack)	Renders the UI; handles SSR + client-side routing; fetches data via TanStack Query.
API Server (Hono)	Central HTTP API. Routes to feature controllers (analytics, social, playlists, AI, import). Validates session token on every protected route.

Component	Responsibility
Auth Module (better-auth)	Handles Spotify OAuth 2.0 login, issues session tokens, stores encrypted provider credentials.
Analytics Engine	Aggregates scrobbles into top-artists/tracks/albums stats, audio-feature summaries, genre distributions, and timeline charts.
AI Insights Module	Builds prompts from a user's listening history and calls the Claude API. Caches responses per user per week.
Playlist & Social Module	Playlist CRUD, follow/unfollow, leaderboard aggregation, music-match scoring, activity feeds.
Background Worker (Hatchet)	Runs async jobs: scrobble sync, token refresh cron, ZIP import processing, weekly playlist generation.
PostgreSQL (Drizzle ORM)	Persistent store for users, scrobbles, tracks, artists, albums, playlists, social graph, AI cache, and import jobs.
Spotify API	OAuth + recently-played endpoint (50 tracks/call). Source of track, artist, album, and audio-feature metadata.
Claude AI API	Produces Taste DNA profiles, personalised roasts, and music-scene reports from listening-history context.
AWS S3	Stores user-uploaded ZIP archives (Spotify Extended Streaming History format).